33.3 聊天命令（Slash Commands）

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~55k tokens，输出 ~5k tokens（本节）

除了 CLI 终端命令（如 `openclaw gateway` 、 `openclaw models` ），OpenClaw 还有一套完整的**聊天命令**系统——用户在聊天窗口中输入 `/status` 、 `/new` 、 `/compact` 等斜杠命令（Slash Commands），就能控制 Agent 的行为。本节拆解这套命令系统的注册、解析和执行机制。

33.3.1 命令注册中心（ `src/auto-reply/commands-registry.data.ts` ）

命令定义类型

每个聊天命令通过 `ChatCommandDefinition` 类型描述：

```typescript
// src/auto-reply/commands-registry.types.ts
export type ChatCommandDefinition = {
  key: string;              // 唯一标识符，如 "status"、"new"
  nativeName?: string;      // 原生命令名称（用于 Telegram/Discord/Slack 的原生斜杠命令注册）
  description: string;      // 命令描述
  textAliases: string[];    // 文本别名列表，如 ["/status"]、["/new", "/reset"]
  acceptsArgs?: boolean;    // 是否接受参数
  args?: CommandArgDefinition[];  // 参数定义
  argsParsing?: "none" | "positional";  // 参数解析模式
  formatArgs?: (values: CommandArgValues) => string | undefined;
  argsMenu?: CommandArgMenuSpec | "auto";  // 原生平台的参数选择菜单
  scope: CommandScope;      // "text" | "native" | "both"
  category?: CommandCategory; // 分类
};

export type CommandScope = "text" | "native" | "both";
export type CommandCategory =
  | "session"     // 会话管理
  | "options"     // 运行时选项
  | "status"      // 状态查询
  | "management"  // 系统管理
  | "media"       // 媒体控制
  | "tools"       // 工具
  | "docks";      // 通道切换
```

三种命令作用域

作用域	说明	示例
`text`	仅在文本消息中识别（如 WhatsApp 消息 "/compact"）	`/compact` 、 `/bash` 、 `/allowlist`
`native`	仅在原生斜杠命令平台上注册（Telegram/Discord/Slack）	无独占命令
`both`	同时支持文本和原生两种方式	`/status` 、 `/new` 、 `/think` 等大多数命令

> **衍生解释 — 原生斜杠命令（Native Slash Commands）**
>
> Telegram、Discord 和 Slack 都支持"原生斜杠命令"——Bot 在注册时向平台声明自己支持哪些 `/command` ，用户在聊天框输入 `/` 时会看到自动补全菜单。这与在普通消息文本中包含 `/status` 不同——原生命令由平台解析并以特殊格式传递给 Bot，而文本命令需要 OpenClaw 自行从消息文本中匹配。

命令工厂

`defineChatCommand` 是命令定义的工厂函数，自动推导默认值：

```
// src/auto-reply/commands-registry.data.ts（简化）
function defineChatCommand(input: DefineChatCommandInput): ChatCommandDefinition {
  // 根据是否有 nativeName 和 textAlias 推导 scope
  const scope = input.scope ??
    (input.nativeName ? (aliases.length ? "both" : "native") : "text");

  // 有 args 定义 → 接受参数
  const acceptsArgs = input.acceptsArgs ?? Boolean(input.args?.length);

  // 有 args 定义 → 位置参数解析
  const argsParsing = input.argsParsing ?? (input.args?.length ? "positional" : "none");

  return { key, nativeName, description, acceptsArgs, args, argsParsing, ... };
}
```

完整命令注册表

`buildChatCommands()` 函数构建完整的命令列表。以下是所有内置聊天命令的分类汇总：

会话管理（session）

命令	别名	说明	参数
/new	—	开始新会话	[指令]（可选，传递给新会话的初始指令）
/reset	—	重置当前会话	[指令]
/compact	—	压缩会话上下文	[instructions]（压缩指令）
/stop	—	停止当前运行	无

运行时选项（options）

命令	别名	说明	参数
/think	/thinking 、 /t	设置思考级别	off / minimal / low / medium / high / xhigh
/verbose	/v	切换详细模式	on / off
/reasoning	/reason	切换推理可见性	on / off / stream
/elevated	/elev	切换提升模式	on / off / ask / full
/model	—	查看/设置模型	[model-id]

命令	别名	说明	参数
`/models`	—	列出模型提供商	—
`/usage`	—	使用量统计	`off / tokens / full / cost`
`/queue`	—	调整队列设置	`steer / interrupt / followup / ...`
`/exec`	—	设置执行参数	`host=... security=... ask=...`

状态查询（status）

命令	别名	说明
`/status`	—	显示当前状态（模型、会话、Token 使用量等）
`/help`	—	显示可用命令
`/commands`	—	列出所有斜杠命令
`/whoami`	`/id`	显示发送者身份
`/context`	—	解释上下文构建方式

管理（management）

命令	说明
`/allowlist`	管理允许列表
`/approve`	审批执行请求
`/config`	查看/设置配置（ `show / get / set / unset` ）
`/debug`	设置运行时调试覆盖
`/subagents`	管理子 Agent（ `list / stop / log / info / send` ）
`/activation`	设置群组激活模式（ `mention / always` ）
`/send`	设置发送策略（ `on / off / inherit` ）

工具（tools）

命令	说明
`/skill`	按名称运行技能
`/bash`	执行主机 Shell 命令（仅限文本模式）
`/restart`	重启 OpenClaw

媒体（media）

命令	说明
`/tts`	控制文本转语音（on/off/status/provider/limit/summary/audio/help）

通道切换（docks）

通道 Dock 命令是**动态生成**的——根据已注册的通道插件中支持原生命令的 Dock，自动创建 `/dock-<id>` 命令：

```
...listChannelDocks()
  .filter((dock) => dock.capabilities.nativeCommands)
  .map((dock) => defineDockCommand(dock)),
```

别名注册

部分命令还有快捷别名：

```
registerAlias(commands, "whoami", "/id");
registerAlias(commands, "think", "/thinking", "/t");
registerAlias(commands, "verbose", "/v");
registerAlias(commands, "reasoning", "/reason");
registerAlias(commands, "elevated", "/elev");
```

`registerAlias` 用 Set 去重，确保不会注册重复的别名。

注册表校验

`assertCommandRegistry` 在命令构建完成后进行严格的完整性校验：

- **唯一性**： `key` 不能重复， `nativeName` 不能重复， `textAlias` 不能重复
- **一致性**： `text` 作用域的命令不能有 `nativeName` ，但必须有 `textAlias` ； `native` 作用域的命令不能有 `textAlias`
- **格式**：所有 `textAlias` 必须以 `/` 开头

缓存策略

命令列表使用单例缓存，通过插件注册表版本控制失效：

```
let cachedCommands: ChatCommandDefinition[] | null = null;
let cachedRegistry: ReturnType<typeof getActivePluginRegistry> | null = null;

export function getChatCommands(): ChatCommandDefinition[] {
  const registry = getActivePluginRegistry();
  if (cachedCommands && registry === cachedRegistry) {
    return cachedCommands;  // 缓存命中
  }
  const commands = buildChatCommands();
  cachedCommands = commands;
  cachedRegistry = registry;
  return commands;
}
```

当插件注册表发生变化时（如加载新插件），缓存自动失效并重建命令列表。

33.3.2 命令识别与分发

命令检测

OpenClaw 的命令检测分为两种模式：

1. **精确匹配**（Exact Match）— 消息整体是一个命令（如 "/status"）
2. **内联检测**（Inline Detection）— 消息中包含命令标记（如 "hey /status"）

内联 `/status` 的检测用正则表达式：

```
// src/auto-reply/reply/reply-inline.ts
const INLINE_STATUS_RE = /(?:^|\s)\/status(?=$|\s|:)(?:\s*:\s*)?/gi;
```

对于允许列表内的发送者，内联 `/status` 会被立即执行并从消息文本中剥离（让模型看不到命令文本）。对于未授权发送者，内联命令被保留为普通文本。

命令执行流程

```
用户消息 "/compact focus on decisions"
    │
    ├── 命令检测 → 匹配 /compact
    │
    ├── 权限检查
    │   ├── 所有者（Owner）→ 允许
    │   ├── 允许列表成员 → 允许
    │   └── 未授权 → 拒绝（仅 /status 允许所有人）
    │
    ├── 参数解析 → { instructions: "focus on decisions" }
    │
    ├── 分发到处理器
    │   ├── /status → commands-status.ts → buildStatusMessage()
    │   ├── /new, /reset → commands-core.ts → 重置会话 + 触发 Hooks
    │   ├── /compact → commands-compact.ts → 调用压缩流水线
    │   ├── /think, /verbose, ... → 更新会话选项
    │   └── /bash → commands-bash.ts → 执行 Shell 命令
    │
    └── 发送回复（文本 / 原生命令响应）
```

`/status` — 状态报告

`/status` 是最常用的聊天命令，它生成一份详细的状态报告，包括：

- 当前模型与提供商
- 会话 Token 使用量和上下文窗口占比
- Agent 配置（思考级别、详细模式、推理模式等）
- 活跃的子 Agent
- 已启用的工具和技能
- 通道连接状态
- 费用统计（如果可用）

状态报告按分类组织，对于授权用户和未授权用户显示不同的信息量。

`/new` 与 `/reset` — 会话重置

`/new` 和 `/reset` 在功能上等价——重置当前会话并开始新的对话。重置过程会：

1. 触发内部钩子（`triggerInternalHook`），允许注册的钩子（如 session-memory）在重置前保存上下文
2. 清除 Token 度量指标
3. 保留会话级覆盖选项（per-session overrides）
4. 如果是"裸重置"（不带参数），触发问候语 Prompt

默认的重置触发词配置在 `config/sessions/types.ts`：

```
export const DEFAULT_RESET_TRIGGER = "/new";
export const DEFAULT_RESET_TRIGGERS = ["/new", "/reset"];
```

`/compact` — 上下文压缩

`/compact` 触发会话上下文压缩流水线，可以附带压缩指令（如 `/compact focus on decisions`）。这是个受限命令——只允许所有者或授权发送者执行，未授权用户的请求会被静默忽略并记录日志。

`/bash` — Shell 命令执行

`/bash` 命令允许直接在宿主机上执行 Shell 命令：

```
defineChatCommand({
  key: "bash",
  description: "Run host shell commands (host-only).",
  textAlias: "/bash",
  scope: "text",        // 仅文本模式（安全考虑，不注册为原生命令）
  category: "tools",
  args: [{ name: "command", type: "string", captureRemaining: true }],
});
```

`/bash` 被有意限制为 `text` 作用域——它不会在 Telegram、Discord、Slack 的原生命令菜单中出现，减少意外执行的风险。执行前需要在配置中启用 `commands.bash=true`。

参数解析系统

聊天命令支持两种参数解析模式：

模式	说明	示例
`positional`	按位置解析参数	`/think medium` → `{ level: "medium" }`
`none`	不自动解析，交给命令自行处理	`/config set agents.defaults.model gpt-4`

参数定义支持丰富的类型和约束：

```
type CommandArgDefinition = {
  name: string;
  description: string;
  type: "string" | "number" | "boolean";
  required?: boolean;
  choices?: CommandArgChoice[] | CommandArgChoicesProvider; // 静态选项或动态生成
  captureRemaining?: boolean;  // 捕获剩余所有文本
};
```

`choices` 字段可以是静态数组，也可以是一个函数——例如 `/think` 的选项列表根据当前模型提供商动态计算（不同模型支持不同的思考级别）：

```
args: [{
  name: "level",
  choices: ({ provider, model }) => listThinkingLevels(provider, model),
}],
```

原生平台的参数菜单

`argsMenu` 字段控制在原生平台上是否为参数提供选择菜单。设为 `"auto"` 时，框架根据 `args` 中的 `choices` 自动生成菜单。也可以手动指定菜单标题和目标参数：

```
argsMenu: {
  arg: "action",
  title: "TTS Actions:\n• On – Enable TTS\n• Off – Disable TTS\n..."
},
```

TUI 集成

TUI（终端用户界面）也集成了聊天命令系统。 `src/tui/commands.ts` 通过 `getSlashCommands()` 将聊天命令转换为 TUI 支持的格式：

```
// src/tui/commands.ts
export function getSlashCommands(options: SlashCommandOptions = {}): SlashCommand[] {
  const commands: SlashCommand[] = [
    { name: "help", description: "Show slash command help" },
    // ... TUI 专属命令
  ];
  // 合并 Gateway 聊天命令
  const gatewayCommands = options.cfg
    ? listChatCommandsForConfig(options.cfg)
    : listChatCommands();
  // ...
}
```

TUI 在用户输入 `/` 时展示命令补全列表，与原生斜杠命令的体验一致。

本节小结

1. **统一命令注册表**：所有聊天命令通过 `ChatCommandDefinition` 统一定义，包括唯一 key、原生名称、文本别名、参数定义和作用域， `assertCommandRegistry` 确保注册表完整一致
2. **三种作用域**： `text` （仅文本消息）、 `native` （仅原生平台）、 `both` （两者兼容），安全敏感命令如 `/bash` 被限制为 `text` 作用域
3. **七个命令分类**：session（会话管理）、options（运行时选项）、status（状态查询）、management（系统管理）、media（媒体）、tools（工具）、docks（通道切换），共计 25+ 内置命令
4. **动态命令生成**：通道 Dock 命令根据已注册的插件动态生成； `/think` 的选项列表根据当前模型动态计算
5. **缓存 + 插件感知**：命令列表使用单例缓存，当插件注册表变化时自动失效重建
6. **权限控制**：大多数命令仅允许所有者和允许列表成员执行；内联命令（如消息中的 `/status` ）对未授权用户保留为普通文本
7. **多平台适配**：同一套命令定义适配 WhatsApp 文本消息、Telegram/Discord/Slack 原生斜杠命令和 TUI 终端界面